

""

Straight Tuck End (STE) carton dieline generator.

This is the folding carton style used for pharma secondary packaging (tablet strips, ampoule boxes, etc.) – the closest single buildable piece of what ArtiosCAD does: turn dimensional inputs into a parametric flat pattern (dieline) with correctly tagged cut/crease lines.

Body panel sequence (left to right, wrapping into a rectangular tube):

GlueFlap -> Front(L) -> Side(D) -> Back(L) -> Side(D)

The glue flap is adhered to the outer face of the last Side panel during manufacture, closing the tube.

Each panel gets top and bottom flaps:

Front/Back -> tuck flaps (trapezoidal, tapered so they slide in)

Side -> dust flaps (rectangular, sit inside the tuck flaps)

Units: millimetres throughout.

""

```

from dataclasses import dataclass, field

@dataclass
class Line:
    x1: float
    y1: float
    x2: float
    y2: float
    kind: str # "cut" or "crease"

@dataclass
class CartonSpec:
    L: float # front/back
    panel width
    D: float # side panel
    depth
    H: float # body height
    thickness: float = 0.3 # board
    caliper (mm) – drives bulge compensation
    glue_flap: float = 15.0 # width of the
    glue flap
    tuck_depth_ratio: float = 0.8 # tuck
    flap depth as fraction of D
    dust_clearance: float = 2.0 # dust
    flap sits this much shorter than tuck flap
    tuck_taper: float = 4.0 # how
    far each side of a tuck flap tapers inward

def bulge_compensation(thickness: float) ->
float:
    """

```

Corrugated/carton board doesn't fold at a mathematical line – it creases with a radius roughly proportional to caliper. Real dieline software adds this allowance to each crease so the ASSEMBLED box hits the target dimensions, not the flat pattern. Rule of thumb: ~2x board thickness.

```
"""
    return 2 * thickness

def generate_tuck_end_carton(spec:
    CartonSpec) -> list[Line]:
    lines: list[Line] = []
    b = bulge_compensation(spec.thickness)

    # --- Horizontal panel boundaries (x-
axis) ---
    # Order: GlueFlap | Front | Side | Back
| Side
    x0 = 0.0
    x1 = x0 + spec.glue_flap          # end
of glue flap / start of Front
    x2 = x1 + spec.L + b              # end
of Front / start of Side
    x3 = x2 + spec.D + b              # end
of Side / start of Back
    x4 = x3 + spec.L + b              # end
of Back / start of Side
    x5 = x4 + spec.D + b              # end
of last Side (outer edge)

    total_width = x5
```

```

        y_top = spec.H          # body spans
y=0 (bottom) to y=H (top) before flaps
        y_bottom = 0.0

        # --- Body outline: cut lines around
the whole flat pattern perimeter,
        #      crease lines at each internal
panel boundary ---
        lines.append(Line(x0, y_bottom, x5,
y_bottom, "cut")) # bottom edge (before
flaps attach)
        lines.append(Line(x0, y_top, x5, y_top,
"cut"))          # top edge (before flaps
attach)
        lines.append(Line(x0, y_bottom, x0,
y_top, "cut"))   # left edge (glue flap
outer edge)
        lines.append(Line(x5, y_bottom, x5,
y_top, "cut"))   # right edge (last side
outer edge)

        for x in (x1, x2, x3, x4):
            lines.append(Line(x, y_bottom, x,
y_top, "crease"))

        # --- Flap generator (shared by top and
bottom, mirrored in y) ---
        def add_flaps(y_body_edge: float,
direction: int):
            """direction: +1 = flaps extend
upward (top), -1 = downward (bottom)"""

            # Front tuck flap (trapezoid,
tapered both sides)

```

```

        fd = spec.D *
spec.tuck_depth_ratio # tuck flap depth
        y_far = y_body_edge + direction *
fd
        # fold line (crease) at panel/flap
boundary
        lines.append(Line(x1, y_body_edge,
x2, y_body_edge, "crease"))
        # tapered sides (cut) + outer edge
(cut)
        lines.append(Line(x1, y_body_edge,
x1 + spec.tuck_taper, y_far, "cut"))
        lines.append(Line(x2, y_body_edge,
x2 - spec.tuck_taper, y_far, "cut"))
        lines.append(Line(x1 +
spec.tuck_taper, y_far, x2 -
spec.tuck_taper, y_far, "cut"))

        # Back tuck flap (identical
geometry, offset to Back panel)
        y_far_back = y_body_edge +
direction * fd
        lines.append(Line(x3, y_body_edge,
x4, y_body_edge, "crease"))
        lines.append(Line(x3, y_body_edge,
x3 + spec.tuck_taper, y_far_back, "cut"))
        lines.append(Line(x4, y_body_edge,
x4 - spec.tuck_taper, y_far_back, "cut"))
        lines.append(Line(x3 +
spec.tuck_taper, y_far_back, x4 -
spec.tuck_taper, y_far_back, "cut"))

        # Dust flaps on both Side panels –
plain rectangles, shorter than tuck flaps

```

```

        dust_depth = fd -
spec.dust_clearance
        y_far_dust = y_body_edge +
direction * dust_depth

        # Side 1 (between x2 and x3)
        lines.append(Line(x2, y_body_edge,
x3, y_body_edge, "crease"))
        lines.append(Line(x2, y_body_edge,
x2, y_far_dust, "cut"))
        lines.append(Line(x3, y_body_edge,
x3, y_far_dust, "cut"))
        lines.append(Line(x2, y_far_dust,
x3, y_far_dust, "cut"))

        # Side 2 (between x4 and x5)
        lines.append(Line(x4, y_body_edge,
x5, y_body_edge, "crease"))
        lines.append(Line(x4, y_body_edge,
x4, y_far_dust, "cut"))
        lines.append(Line(x5, y_body_edge,
x5, y_far_dust, "cut"))
        lines.append(Line(x4, y_far_dust,
x5, y_far_dust, "cut"))

        add_flaps(y_top, +1)      # top flap
set
        add_flaps(y_bottom, -1)  # bottom flap
set (straight tuck = same style both ends)

        return lines

if __name__ == "__main__":

```

```
spec = CartonSpec(L=60, D=30, H=100)
result = generate_tuck_end_carton(spec)
print(f"Generated {len(result)} lines")
for ln in result[:5]:
    print(ln)
```